实验报告

姓名：陈秀萍

学号：2026001406

第一题

本题分别使用 R 和 RcppArmadillo 实现 3×3 灰度图像卷积，并通过高斯模糊、锐化和边缘检测三个卷积核进行验证。两种实现的输出在浮点计算误差范围内一致；在 256×256 图像上，C++ 实现的平均运行时间明显短于 R 实现。

1 实现方法

输入图像以矩阵表示，每个元素均位于区间 [0, 1]。程序对每个像素依次访问其 3×3 邻域，并按照下式直接使用卷积核，不对卷积核进行翻转。

$$C_{ij} = \sum_{a=-1}^{1} \sum_{b=-1}^{1} A_{i+a,\, j+b} K_{a+2,\, b+2}$$

边界采用镜像延拓。对于 C++ 的零起始下标，越过左侧或上侧边界的 −1 映射到 0，越过右侧或下侧边界的 length 映射到 length−1；R 版本使用与一号起始下标对应的规则。卷积完成后，将小于 0 的结果截断为 0，将大于 1 的结果截断为 1。两种实现均使用循环完成相同的运算。

2 数值验证

使用同一幅可重复生成的灰度测试图像，对三个卷积核分别计算 R 与 C++ 输出之间的最大绝对误差，结果见表 1。

表 1 R 与 C++ 卷积结果的最大绝对误差

卷积核	最大绝对误差
Gaussian	0.000000e+00
Sharpen	4.440892e−16
Edge	0.000000e+00

三个卷积核对应的最大绝对误差均不超过 4.44×10^{-16} ，远小于验证阈值 10^{-10} 。因此，可以认为 R 与 C++ 两种实现给出了相同的数值结果。

3 运行时间比较

计时使用 256×256 的同一幅图像和 Gaussian 卷积核。C++ 编译、数据生成、预热和绘图均不计入运行时间；每种方法重复运行 10 次，报告单次调用的平均耗时。

表 2 R 与 C++ 实现的运行时间

方法	平均时间（秒）	重复次数	图像尺寸
C++	0.0003	10	256×256
R	0.2524	10	256×256

按照本次计时结果，R 实现的平均耗时约为 C++ 实现的 841 倍，即 C++ 实现约快 841 倍。主要原因是 R 版本在解释器中执行多层循环，而 C++ 版本将循环编译为机器代码，因此更适合像素级重复计算。

4 图像处理结果

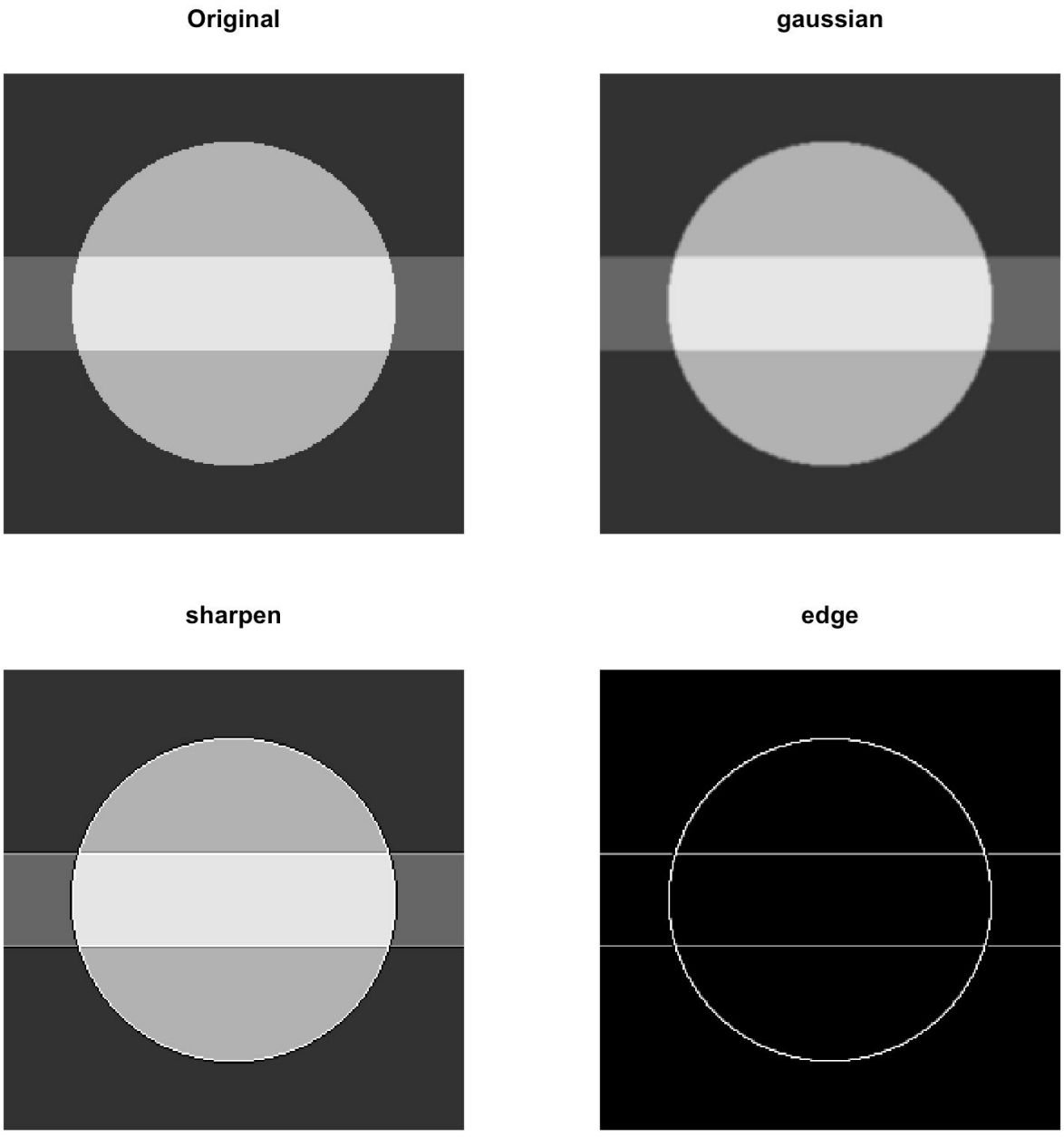


图 1 原始图像及 Gaussian、Sharpen 和 Edge 卷积结果

Gaussian 卷积对图像进行平滑，使区域边界变得柔和；Sharpen 卷积增强了局部灰度变化，使边缘更加突出；Edge 卷积主要保留灰度变化显著的位置，从而突出图像轮廓。所有输出像素均被限制在 [0, 1] 内，且输出矩阵尺寸与原图一致。

5 小结

本题完成了 R 与 RcppArmadillo 两个版本的 3×3 图像卷积。数值验证表明两种实现一致，计时结果表明 C++ 在多层像素循环任务中具有明显的效率优势。

第二题

本题使用 RcppArmadillo 计算多元正态分布的对数概率密度。实现采用下三角 Cholesky 分解和三角方程求解，避免显式计算协方差矩阵的逆矩阵和行列式。四组测试的计算值均与参考值一致，验证结果通过。

1 计算目标

设 x 和 μ 为 p 维向量，Σ 为 p×p 对称正定协方差矩阵。多元正态分布的对数概率密度为：

$$\log f(x) = -\frac{p}{2}\log(2\pi) - \frac{1}{2}\log\det\Sigma - \frac{1}{2}(x-\mu)^{\mathsf{T}}\Sigma^{-1}(x-\mu)$$

2 实现方法

程序首先计算下三角 Cholesky 分解 $\Sigma=LL^{\mathsf{T}}$，其中 L 为下三角矩阵。随后通过求解三角方程 $Lv=x-\mu$ 得到向量 v，并利用 $v^{\mathsf{T}}v$ 计算二次型。对数行列式项由 L 的对角元素直接得到，因此不需要先计算 det(Σ) 再取对数。最终使用下式返回对数概率密度：

$$\Sigma = LL^{\mathsf{T}}, \quad Lv = x-\mu, \quad \log f(x) = -\frac{p}{2}\log(2\pi) - \sum_{i=1}^{p}\log L_{ii} - \frac{1}{2}v^{\mathsf{T}}v$$

这种计算方式避免了显式求逆带来的额外计算量和数值误差，也避免了行列式数值过大或过小导致的上溢与下溢问题。

3 测试与验证

测试包括一维正态分布、对角协方差矩阵、非对角协方差矩阵，以及额外设计的一组自定义测试。R 脚本使用独立计算得到的参考值进行比较，并要求绝对误差小于 10^{-10} 。

表 3 多元正态对数概率密度测试结果

测试情形	C++ 计算值	参考值	结果
一维正态分布	−1.504404	−1.504404	通过
对角协方差矩阵	−5.079362	−5.079362	通过
非对角协方差矩阵	−4.096459	−4.096459	通过
自定义测试	−2.810832	−2.810832	通过

四组测试中，C++ 计算值与参考值在显示精度下完全一致，并且所有测试均满足绝对误差小于 10^{-10} 的要求。这说明 Cholesky 分解、三角方程求解及对数密度计算均实现正确。该实现没有显式计算 Σ^{-1} 或 $\det(\Sigma)$，在保证结果正确的同时提高了数值稳定性。

第三题

本题在已知初始状态的线性高斯状态空间模型中实现 Kalman smoothing。程序先完成前向 Kalman filtering 并保存后向递推所需的预测量和过滤量，再进行后向平滑。输出维度、终点一致性及单观测情形均通过检查；本次模拟中，平滑估计的位置均方误差明显小于过滤估计。

1 状态空间模型

状态向量包含二维位置和二维速度，观测向量为二维位置。状态转移噪声和观测噪声相互独立，sigmaQ 与 sigmaR 分别表示两类噪声的标准差。模型写为：

$$z_t = Az_{t-1} + \varepsilon_t, \qquad y_t = Cz_t + \eta_t, \qquad \varepsilon_t \sim N(0, \sigma_Q^2 I_4), \quad \eta_t \sim N(0, \sigma_R^2 I_2)$$

2 前向过滤

前向步骤从已知初始状态和零初始协方差出发，依次计算每个时刻的预测均值、预测协方差、Kalman 增益、过滤均值和过滤协方差。相关量全部保存，以供后向平滑使用。主要递推关系为：

$$\hat{z}_{t\mid t-1} = A\hat{z}_{t-1\mid t-1}, \qquad P_{t\mid t-1} = AP_{t-1\mid t-1}A^{\mathsf{T}} + \sigma_Q^2 I_4$$

$$K_t = P_{t\mid t-1}C^{\mathsf{T}}(CP_{t\mid t-1}C^{\mathsf{T}} + \sigma_R^2 I_2)^{-1}, \qquad \hat{z}_{t\mid t} = \hat{z}_{t\mid t-1} + K_t(y_t - C\hat{z}_{t\mid t-1})$$

3 后向平滑

后向步骤以最后一个时刻的过滤结果作为平滑初值，从 T−1 时刻向前递推。平滑增益和状态均值的更新公式为：

$$J_t = P_{t\mid t}A^{\mathsf{T}}(P_{t+1\mid t})^{-1}, \qquad \hat{z}_{t\mid T} = \hat{z}_{t\mid t} + J_t(\hat{z}_{t+1\mid T} - \hat{z}_{t+1\mid t})$$

Filtering 在时刻 t 仅利用从第 1 个时刻到第 t 个时刻的观测，而 smoothing 在估计同一状态时还利用后续观测。因此，smoothing 能够根据完整时间序列对早期状态进行修正。

4 数值验证

模拟设置 T=100，过程噪声标准差为 0.2，观测噪声标准差为 0.8。程序检查平滑结果维度为 4×100、所有元素均为有限值、最后一个时刻的平滑均值与过滤均值一致，并验证单个观测时 filtering 与 smoothing 一致。

表 1 Filtering 与 Smoothing 的位置估计误差

方法	位置 MSE	相对 Filtering 的变化
Filtering	0.25161834	基准
Smoothing	0.06381335	降低约 74.64%

本次模拟中，Smoothing 的位置 MSE 由 0.25161834 降至 0.06381335，下降约 74.64%。这一结果说明利用未来观测对历史状态进行修正能够明显提高本次路径的位置估计精度。单次模拟并不要求 smoothing 的误差必然更小，但本次结果与其利用更多信息的特点一致。

5 路径估计结果

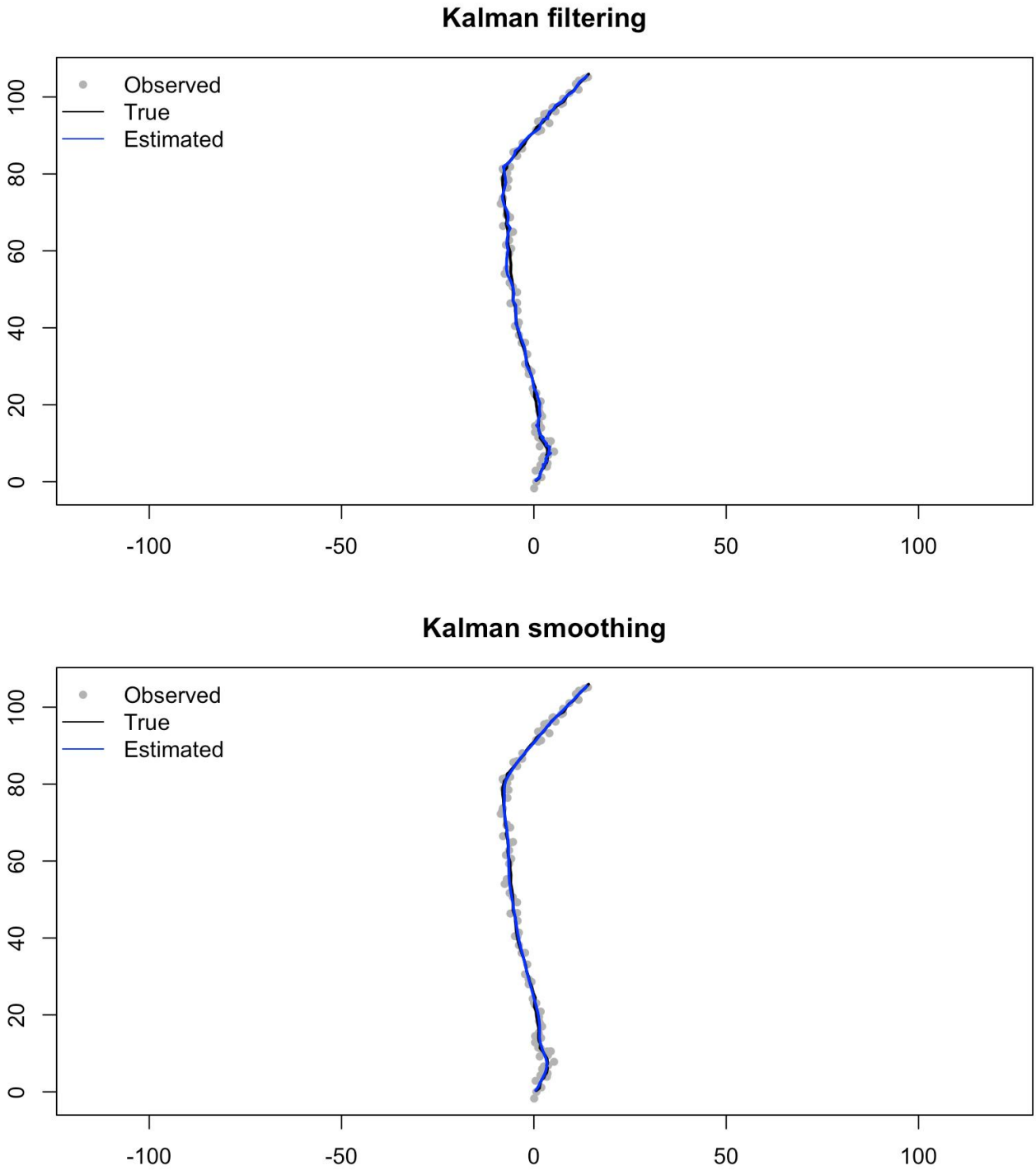


图 1 Kalman filtering 与 smoothing 的路径估计比较

图中灰色散点为带噪声的观测位置，黑色曲线为真实路径，蓝色曲线为相应的估计路径。Filtering 路径会随当前观测逐步更新；Smoothing 路径结合完整观测序列后更加接近真实轨迹，局部波动也相对较小。

6 小结

本题完成了 Kalman filtering 的前向计算和 Rauch–Tung–Striebel 后向平滑。各项程序检查均通过，且本次模拟中 smoothing 获得了更低的位置估计误差。

第四题

本题分别实现 Ridge regression 的三种留一交叉验证算法，以及基于广义奇异值分解的 Tikhonov regularization。三种 Ridge 算法得到相同的交叉验证结果和最优参数；SVD 方法速度最快。Tikhonov 系数、拟合值及退化为 Ridge 时的结果均与参考计算一致。

1 Ridge Regression 模型

在不加入截距且不对变量进行标准化的条件下，Ridge regression 通过二次惩罚限制系数规模。其目标函数为：

$$\widehat{\beta}_\lambda = \arg\min_\beta \{\|y - X\beta\|_2^2 + \lambda\|\beta\|_2^2\}$$

实现过程中不显式计算矩阵逆，而是使用线性方程求解或分解结果完成系数、拟合值和杠杆值的计算。候选参数包含 20 个 lambda 值，样本量 n=200，变量数 p=20。

2 三种 LOOCV 计算方法

2.1 Direct 方法

对每一个 lambda，依次删除第 i 个观测，在其余 n−1 个观测上重新拟合模型，再计算被删除观测的预测误差。该方法定义直接、便于作为正确性基准，但需要对每个 lambda 重复拟合 n 次。

2.2 Leverage 方法

Ridge 拟合值可写为线性平滑形式。利用平滑矩阵对角元即可从全样本残差得到留一残差，避免逐个删除观测：

$$S_\lambda = X(X^\top X + \lambda I_p)^{-1} X^\top$$

$$\mathrm{CV}(\lambda) = \sum_{i=1}^{n} \left(\frac{y_i - \widehat{y}_i}{1 - s_{ii,\lambda}}\right)^2$$

2.3 SVD 方法

程序只对设计矩阵 X 进行一次奇异值分解，并在全部 lambda 上复用分解结果。每个参数只需更新奇异值对应的收缩因子，因此进一步减少了重复分解和线性求解。

$$X = UDV^{\mathsf{T}}, \qquad \widehat{y}_\lambda = U\,\mathrm{diag}\!\left(\frac{d_j^2}{d_j^2+\lambda}\right)U^{\mathsf{T}}y$$

$$s_{ii,\lambda} = \sum_j u_{ij}^2 \frac{d_j^2}{d_j^2+\lambda}$$

3 Ridge Regression 数值结果

以 Direct 方法为基准，Leverage 与 SVD 方法的交叉验证误差仅存在浮点舍入级别的差异。三种方法均选择 lambda=0.6951928。

表 1 三种 Ridge LOOCV 方法的一致性

方法	最大绝对误差	选择的 lambda
Direct	0.000000e+00	0.6951928
Leverage	2.273737e−13	0.6951928
SVD	3.694822e−13	0.6951928

Leverage 和 SVD 的最大绝对误差均小于 4×10^{-13}，远低于数值验证阈值，说明三种实现得到相同的 LOOCV 曲线。

表 2 全部 20 个 lambda 的运行时间

方法	中位时间 秒	重复次数	每批调用	n	p
Direct	0.104000	5	1	200	20
Leverage	0.001140	5	100	200	20
SVD	0.000174	5	500	200	20

Direct 方法每次计算耗时 0.104000 秒；Leverage 方法为 0.001140 秒，约快 91.2 倍；SVD 方法为 0.000174 秒，约快 597.7 倍。SVD 还比 Leverage 约快 6.6 倍。计时包含各方法完成全部 lambda 所需的预处理和分解，不包含数据生成与绘图。

4 Ridge LOOCV 曲线

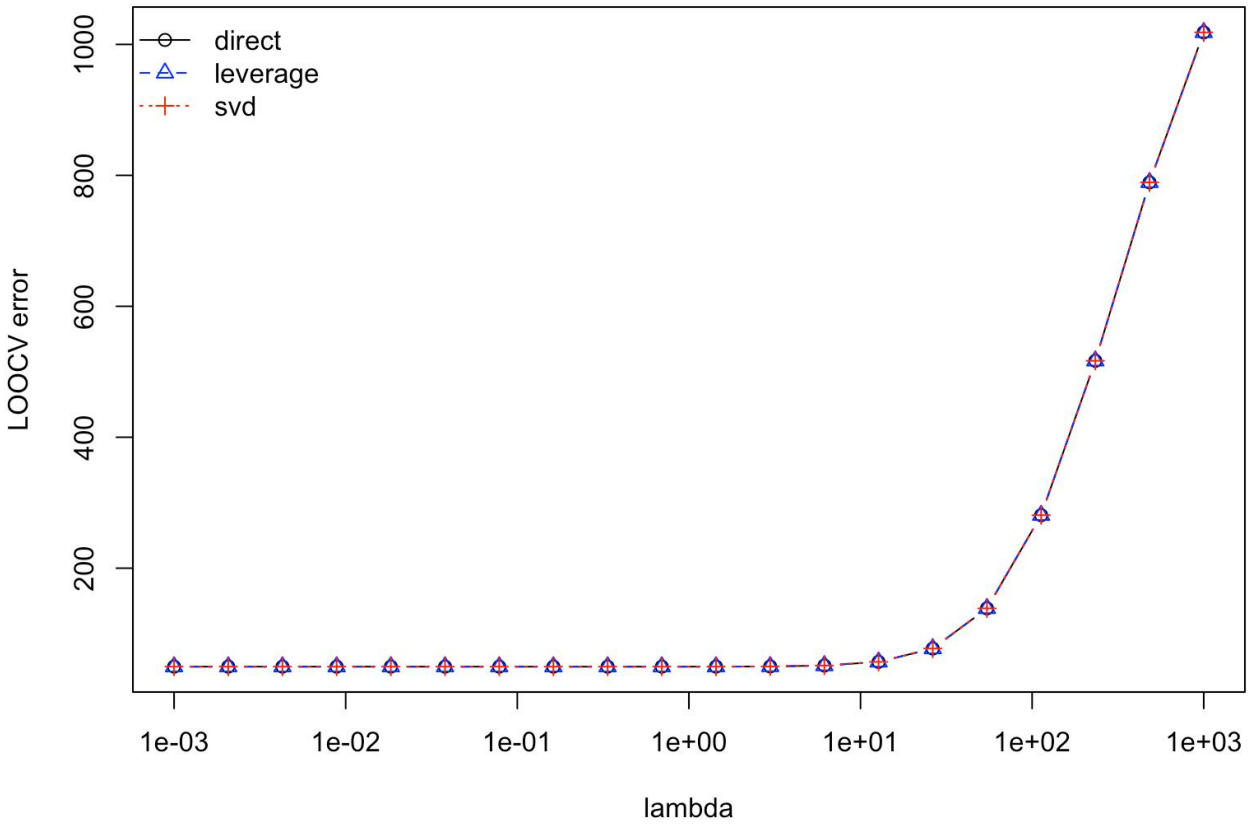


图 1 三种方法得到的 Ridge LOOCV 误差曲线

横轴为对数尺度下的 lambda，纵轴为 LOOCV error。三条曲线在绘图精度下重合，并在 lambda=0.6951928 处达到相同的最小值，进一步验证了三种算法的一致性。

5 Tikhonov Regularization

Tikhonov regularization 将 Ridge 的单位矩阵惩罚推广为一般矩阵 B，使不同系数方向可以受到不同强度的约束。其目标函数为：

$$\widehat{\beta}_\lambda = \arg\min_\beta \{\|y - X\beta\|_2^2 + \lambda\|B\beta\|_2^2\}$$

5.1 GSVD 计算

程序使用辅助函数返回的 U、Z、alpha 和 beta 构造解，并预先计算 $U^T y$，在所有 lambda 上复用。为避免与回归系数 beta 混淆，下式将 GSVD 输出 beta 记为 gamma。系数与拟合值的计算式为：

$$X = U\,\mathrm{diag}(\alpha_i)Z^{-1}, \qquad B = U_2\,\mathrm{diag}(\gamma_i)Z^{-1}$$

$$\widehat{\beta}_\lambda = \sum_i \frac{\alpha_i(u_i^T y)}{\alpha_i^2 + \lambda\gamma_i^2} z_i, \qquad \widehat{y}_\lambda = \sum_i \frac{\alpha_i^2}{\alpha_i^2 + \lambda\gamma_i^2}(u_i^T y)u_i$$

这一实现直接使用 GSVD 的分解结构，没有通过普通正规方程替代指定算法。拟合值既由 GSVD 展开式直接得到，也用 X 乘以系数再次计算，以验证内部一致性。

5.2 数值验证

表 3 Tikhonov 与参考解的一致性检查

检查项目	最大绝对误差
系数与 solve 参考解	2.664535e−15
拟合值与参考解	7.993606e−15
拟合值与 Xβ 的一致性	5.551115e−15
B=I 时与 Ridge 的一致性	4.440892e−15

四项误差均小于 8×10^{-15}，处于双精度浮点运算的舍入范围内。因此，GSVD 系数和拟合值与 solve 参考结果一致，内部拟合关系成立，并且 B=I 时正确退化为 Ridge regression。

6 小结

本题完成了 Direct、Leverage 和 SVD 三种 Ridge LOOCV 实现。三者选择相同的最优 lambda，数值结果一致，而复用单次分解的 SVD 方法效率最高。基于 GSVD 的 Tikhonov 实现也通过全部参考解与一致性检查。